LAB ASSIGNMENT

Implementation of RNN-Based Text Classification

Course: Artificial Neural Networks and Deep Learning

Lab: Recurrent Neural Network (RNN) Classifier

Assignment Type: Individual Practical Assignment

Maximum Marks: 20

---

1. Aim

To implement a **Recurrent Neural Network (RNN)** model using TensorFlow/Keras for **binary text classification** and evaluate its performance on the IMDB movie-review dataset.

---

2. Learning Objectives

After completing this laboratory assignment, students will be able to:

- Understand the basic architecture and working of an RNN.
- Perform text preprocessing and sequence padding.
- Use an **Embedding layer** for text representation.
- Build an RNN-based binary classification model.
- Train and evaluate an RNN model using TensorFlow/Keras.
- Visualize training and validation performance.
- Analyze the effect of different hyperparameters on model performance.

---

3. Problem Statement

You are required to develop an **RNN-based sentiment classification system** that automatically classifies movie reviews as:

- **Positive – 1**
- **Negative – 0**

Use the **IMDB Movie Reviews dataset** available through `tensorflow.keras.datasets`.

Your model should contain at least:

1. An **Embedding layer**
2. A **SimpleRNN layer**
3. A **Dense output layer with sigmoid activation**

4. Tasks to be Performed

Task 1 – Load and Explore the Dataset

Load the IMDB dataset using TensorFlow/Keras.

Use:

- Vocabulary size = **10,000**
- Maximum sequence length = **200**

Display:

- Number of training samples
- Number of testing samples
- Shape of training data
- Shape of testing data
- Number of positive and negative reviews

Task 2 – Data Preprocessing

Perform the following preprocessing steps:

1. Load the IMDB dataset.
2. Limit the vocabulary to the 10,000 most frequent words.
3. Convert reviews into numerical sequences.
4. Apply padding/truncation.
5. Ensure that all reviews have the same length of 200.

Display the shape of the final training and testing datasets.

Task 3 – Build the RNN Classifier

Construct an RNN model using the following basic architecture:

Input → Embedding → SimpleRNN → Dense → Output

Suggested architecture:

- Embedding dimension = 128
- RNN units = 64
- Output activation = sigmoid

Students must display the model summary.

Task 4 – Compile and Train the Model

Compile the model using:

- Optimizer: **Adam**
- Loss function: **Binary Cross-Entropy**
- Evaluation metric: **Accuracy**

Train the model for at least **5 epochs**.

Use:

- Batch size = 64
- Validation split = 20%

Record:

- Training accuracy
- Validation accuracy
- Training loss
- Validation loss

Task 5 – Evaluate the Model

Evaluate the trained model using the test dataset.

Report:

- Test loss
- Test accuracy

Answer the following:

Q1. What is the final test accuracy of your RNN model?

Q2. Is the test accuracy higher or lower than the training accuracy? Explain why.

---

5. Task 6 – Performance Visualization

Generate the following two graphs:

Graph 1 – Accuracy

Plot:

- Training Accuracy
- Validation Accuracy

against the number of epochs.

Graph 2 – Loss

Plot:

- Training Loss
- Validation Loss

against the number of epochs.

Students must interpret both graphs.

---

6. Task 7 – Hyperparameter Experimentation

Students must perform the following experiments.

Experiment A – RNN Units

Train the model using:

Experiment	RNN Units
A1	32
A2	64
A3	128

Record the test accuracy.

Experiment B – Number of Epochs

Use:

Experiment	Epochs
B1	3
B2	5
B3	10

Record the training accuracy, validation accuracy and test accuracy.

Experiment C – Batch Size

Use:

Experiment	Batch Size
C1	32
C2	64
C3	128

Compare the results.

---

7. Results Table

Students must complete the following table.

Experiment	Parameter	Training Accuracy	Validation Accuracy	Test Accuracy
A1	RNN = 32			
A2	RNN = 64			
A3	RNN = 128			
B1	Epochs = 3			
B2	Epochs = 5			
B3	Epochs = 10			
C1	Batch = 32			
C2	Batch = 64			
C3	Batch = 128			

---

8. Task 8 – Text Prediction

Create a function that accepts a new movie review and predicts whether the review is **Positive** or **Negative**.

Test your model with at least **five new reviews**.

Use the following format:

Review	Actual/Expected Sentiment	Predicted Sentiment	Probability
Review 1	Positive		
Review 2	Negative		
Review 3	Positive		

Review	Actual/Expected Sentiment	Predicted Sentiment	Probability
Review 4	Negative		
Review 5	Positive		

Students should write their own five test reviews.

---

9. Task 9 – Challenge Activity

Modify the original RNN model by adding a second RNN layer.

For example:

Embedding → SimpleRNN → SimpleRNN → Dense

Investigate the purpose of:

```
return_sequences=True
```

in the first RNN layer.

Compare the single-layer and two-layer RNN models.

Answer:

1. Which model gives better test accuracy?
2. Which model takes longer to train?
3. Does adding another RNN layer always improve performance?
4. What is the purpose of `return_sequences=True`?

---

10. Discussion Questions

Answer the following questions in your lab report:

1. What is an RNN and why is it suitable for text classification?
2. What is the purpose of the Embedding layer?
3. Why is padding required?
4. Why is sigmoid activation used in the output layer?
5. Why is binary cross-entropy used as the loss function?
6. What is the vanishing-gradient problem?
7. What happens when the number of RNN units is increased?
8. What is overfitting?
9. How can overfitting be reduced in an RNN?
10. What is the difference between SimpleRNN, LSTM and GRU?

---

11. Expected Output

The submitted notebook should contain:

- Dataset information
- Preprocessing output
- Model summary
- Training results
- Test accuracy
- Accuracy graph
- Loss graph
- Hyperparameter experiment results
- Five prediction examples
- Comparison of single-layer and two-layer RNN
- Answers to discussion questions
- Final conclusion

---

12. Conclusion

Write a brief conclusion of approximately **150–200 words** explaining:

- What you learned from the experiment.
- How the RNN performed.
- Which hyperparameters produced the best result.
- Whether increasing RNN units or epochs improved performance.
- Any limitations observed in the model.
- How LSTM/GRU could potentially improve the classification system.

---

13. Submission Guidelines

Students must submit:

1. **Jupyter/Google Colab Notebook (`.ipynb`)**
2. **Lab report in PDF**
3. All graphs and experimental results
4. Proper comments in the Python code

Important Instructions

- Students must write and execute their own code.
- Screenshots of code without execution/results will not be accepted.
- All graphs must have proper titles and axis labels.
- Students must explain their experimental results.
- Copying another student's notebook will result in zero marks.
- The notebook should execute without errors from beginning to end.

14. Assessment Rubric – 20 Marks

Component	Marks
Dataset loading & preprocessing	3
RNN model implementation	4
Training & evaluation	3
Visualization	2
Hyperparameter experiments	4
Prediction & analysis	2
Viva/Discussion	2
Total	**20**

Viva Questions

1. What is the difference between ANN and RNN?
2. Why does an RNN have a memory?
3. What is a hidden state?
4. What is the role of the Embedding layer?
5. What is sequence padding?
6. Explain `return_sequences=True`.
7. What is the vanishing-gradient problem?
8. Why is LSTM preferred over SimpleRNN for long sequences?
9. What is binary cross-entropy?
10. What happens if the learning rate is too high?
11. What is overfitting?
12. How can dropout be incorporated into an RNN?
13. What is the difference between LSTM and GRU?
14. Why is sigmoid used for binary classification?
15. Does increasing the number of RNN units always improve accuracy?

Deliverable

Submit one Google Colab/Jupyter Notebook containing the complete implementation, results, graphs, experiments, answers, and conclusion.